

DEVOPS PRACTICALS

Complete Step-by-Step Commands and Procedures – Practicals 1 to 12

Topics: Git, Version Control, GitHub, GitLab and Docker

Practical 1: Creation of Git Repository, File Upload, Commit and Repository Management

Aim: To create a Git repository, add a file, commit the changes, and manage the repository.

Step 1: Check Git installation

```
git --version
```

Git must be installed before starting.

Step 2: Configure Git

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --list
```

Configure your name and email. Usually done only once.

Step 3: Create a project folder

```
mkdir MyProject
cd MyProject
```

Step 4: Create a Git repository

```
git init
```

Step 5: Create a file

```
echo "Hello Git" > hello.txt
ls
```

Step 6: Check status

```
git status
```

The file will initially be shown as untracked.

Step 7: Add the file

```
git add hello.txt
```

You can also use `git add .` to add all changed files.

Step 8: Commit

```
git commit -m "Initial commit"
```

Step 9: View commit history

```
git log --oneline
```

Result: Git repository was created successfully, the file was added and committed, and repository status/history were checked.

Practical 2: Demonstration of a Version Control Tool

Aim: To demonstrate Git as a version control tool for tracking file changes.

Step 1: Create project

```
mkdir VersionControl  
cd VersionControl
```

Step 2: Initialize repository

```
git init
```

Step 3: Create a file

```
echo "Version Control using Git" > file.txt
```

Step 4: Check status

```
git status
```

Step 5: Stage file

```
git add file.txt
```

Step 6: Commit first version

```
git commit -m "Added file.txt"
```

Step 7: Modify the file

```
echo "This is the second version" >> file.txt
```

Step 8: Check changes

```
git status
```

Step 9: View exact changes

```
git diff
```

Step 10: Commit second version

```
git add file.txt  
git commit -m "Updated file.txt"
```

Step 11: View history

```
git log --oneline
```

Result: Git successfully tracked different versions of the file and maintained the project history.

Practical 3: Git Branching and Merging through Collaborative Coding

Aim: To create a branch, make changes on the branch, and merge the branch into the main branch.

Step 1: Create repository

```
mkdir BranchProject
cd BranchProject
git init
```

Step 2: Create initial file

```
echo "Main Project" > index.txt
```

Step 3: Commit initial version

```
git add .
git commit -m "Initial commit"
```

Step 4: Create a feature branch

```
git branch feature
git branch
```

Step 5: Switch to feature branch

```
git switch feature
```

If git switch is unavailable, use git checkout feature.

Step 6: Make changes

```
echo "New Feature Added" >> index.txt
```

Step 7: Commit feature

```
git add .
git commit -m "Added new feature"
```

Step 8: Switch to main

```
git switch main
```

If your branch is named master, use git switch master.

Step 9: Merge feature

```
git merge feature
```

Step 10: View history

```
git log --oneline --all
```

Important commands / notes

```
git branch feature
git branch
git switch feature
git merge feature
git branch -d feature
```

Result: A feature branch was created, changes were committed, and the feature branch was merged successfully.

Practical 4: Creating and Committing Changes

Aim: To create changes in a Git repository, stage them, and commit them.

Step 1: Create repository

```
mkdir CommitProject
cd CommitProject
git init
```

Step 2: Create file

```
echo "First version" > demo.txt
```

Step 3: Check status

```
git status
```

Step 4: Stage file

```
git add demo.txt
```

Step 5: Commit

```
git commit -m "Added demo file"
```

Step 6: Modify file

```
echo "Second version" >> demo.txt
```

Step 7: Check status

```
git status
```

Step 8: Stage modified file

```
git add demo.txt
```

Step 9: Commit again

```
git commit -m "Updated demo file"
```

Step 10: View commits

```
git log --oneline
```

Result: Changes were successfully created, staged, and committed.

Practical 5: Implementation of git init, git add, git commit and git status

Aim: To implement the basic Git commands used for creating and saving changes in a repository.

Step 1: Create a folder

```
mkdir GitCommands  
cd GitCommands
```

Step 2: Initialize Git

```
git init
```

Step 3: Create a file

```
echo "Git Commands Practical" > practical.txt
```

Step 4: Check status

```
git status
```

The file appears as untracked.

Step 5: Add the file

```
git add practical.txt
```

Step 6: Check status again

```
git status
```

The file should now appear under changes to be committed.

Step 7: Commit

```
git commit -m "First commit"
```

Step 8: Check final status

```
git status
```

The working tree should be clean.

Result: The basic Git commands git init, git add, git commit and git status were successfully implemented.

Practical 6: Setting Up a Git Repository

Aim: To set up a local Git repository and prepare it for version control.

Step 1: Create project folder

```
mkdir SetupProject  
cd SetupProject
```

Step 2: Initialize repository

```
git init
```

Step 3: Create project files

```
echo "My Git Project" > README.txt  
echo "print(\"Hello World\")" > app.py
```

Step 4: Check repository

```
git status
```

Step 5: Stage all files

```
git add .
```

Step 6: Commit files

```
git commit -m "Initial project setup"
```

Step 7: Check history

```
git log --oneline
```

Result: The Git repository was successfully set up with project files and an initial commit.

Practical 7: GitHub Operations – Push/Pull Changes to GitHub

Aim: To connect a local Git repository to GitHub and push and pull changes.

Step 1: Create a repository on GitHub

Open GitHub → Sign in → New repository → Enter repository name → Create repository

Create an empty repository. Do not initialize with a README if you already have a local project.

Step 2: Create local repository

```
mkdir GitHubProject
cd GitHubProject
git init
```

Step 3: Create and commit a file

```
echo "Hello GitHub" > index.txt
git add .
git commit -m "Initial commit"
```

Step 4: Connect local repository to GitHub

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

Replace USERNAME and REPOSITORY with your GitHub details.

Step 5: Rename branch to main

```
git branch -M main
```

Step 6: Push local changes

```
git push -u origin main
```

GitHub may ask you to authenticate.

Step 7: Make another local change

```
echo "New change" >> index.txt
git add .
git commit -m "Added new change"
```

Step 8: Push the new commit

```
git push
```

Step 9: Pull changes from GitHub

```
git pull origin main
```

Use this when changes have been made on the remote repository.

Important commands / notes

```
Check remote: git remote -v
Push: git push
Pull: git pull origin main
```

Result: Local changes were successfully pushed to GitHub and remote changes were retrieved using pull.

Practical 8: Cloning a Repository

Aim: To copy an existing GitHub repository to the local computer using git clone.

Step 1: Copy the repository URL

Open the GitHub repository → Code → HTTPS → Copy URL

Step 2: Clone repository

```
git clone https://github.com/USERNAME/REPOSITORY.git
```

Replace the URL with the actual repository URL.

Step 3: Enter the cloned folder

```
cd REPOSITORY
```

Replace REPOSITORY with the folder/repository name.

Step 4: View files

```
ls
```

Step 5: Check Git status

```
git status
```

Step 6: Make a change

```
echo "Cloned repository updated" >> update.txt
```

Step 7: Commit the change

```
git add .  
git commit -m "Updated cloned repository"
```

Step 8: Push the change

```
git push
```

Result: The GitHub repository was successfully cloned to the local computer and changes were committed and pushed.

Practical 9: Create and Review a Merge Request on GitLab

Aim: To create a GitLab Merge Request by working on a separate branch and requesting that its changes be merged into the main branch.

Step 1: Create/sign in to GitLab

Open GitLab → Sign in/Create account

Step 2: Create a project

GitLab → New project/repository → Create blank project → Enter name → Create project

Step 3: Clone the project

```
git clone https://gitlab.com/USERNAME/PROJECT.git
cd PROJECT
```

Replace USERNAME and PROJECT with your details.

Step 4: Create a feature branch

```
git switch -c feature
```

This creates and switches to the new branch.

Step 5: Create or modify a file

```
echo "Feature for review" > feature.txt
```

Step 6: Commit the change

```
git add .
git commit -m "Added feature"
```

Step 7: Push feature branch

```
git push -u origin feature
```

Step 8: Create Merge Request

Open GitLab project → Merge requests → New merge request → Source: feature → Target: main → Create merge request

Step 9: Review the Merge Request

Open the Merge Request → Review Changes/Diff → Add comments if required → Approve or request changes

Step 10: Merge

After review, select Merge → Confirm merge

Important commands / notes

```
Branch creation: git switch -c feature
Push branch: git push -u origin feature
```

Result: The feature branch was pushed to GitLab, a Merge Request was created and reviewed, and the changes could be merged into the main branch.

Practical 10: Create a Docker Account Step by Step

Aim: To create a Docker account and prepare Docker Hub for storing and sharing container images.

Step 1: Open Docker website

Open `https://hub.docker.com/`

Use a web browser.

Step 2: Sign up

Click Sign Up → Enter email/username/password → Complete verification

Step 3: Verify email

Open the verification email → Verify your Docker account

Step 4: Sign in

Open Docker Hub → Sign in with your Docker account

Step 5: Create a repository

Docker Hub → Repositories → Create Repository → Enter repository name → Create

Step 6: Install Docker Desktop if required

Download Docker Desktop → Install → Start Docker Desktop

For command-line Docker practicals, Docker Engine/Desktop must be running.

Step 7: Check Docker

```
docker --version
```

Step 8: Login from terminal

```
docker login
```

Enter Docker Hub credentials or follow the displayed authentication flow.

Result: Docker account and Docker Hub repository were successfully created, and Docker was prepared for use.

Practical 11: Dockerfile Instructions and Build an Image for a Sample Web Application

Aim: To study Dockerfile instructions and build a Docker image for a simple web application.

Step 1: Create project folder

```
mkdir DockerWebApp
cd DockerWebApp
```

Step 2: Create a simple HTML file

```
echo "<html><body><h1>Hello from Docker</h1></body></html>" > index.html
```

Step 3: Create Dockerfile

Create a file named Dockerfile with the following content:

Dockerfile content

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Save the file exactly as Dockerfile without a .txt extension.

Step 4: Build the image

```
docker build -t mywebapp .
```

The dot means the current folder is the build context.

Step 5: Check the image

```
docker images
```

Step 6: Run a container

```
docker run -d -p 8080:80 --name mywebcontainer mywebapp
```

Step 7: Open the application

Open browser → <http://localhost:8080>

You should see 'Hello from Docker'.

Step 8: Check running containers

```
docker ps
```

Step 9: Stop the container

```
docker stop mywebcontainer
```

Step 10: Remove the container

```
docker rm mywebcontainer
```

Important commands / notes

Common Dockerfile instructions:

- FROM – specifies the base image
- COPY – copies files into the image
- RUN – executes commands while building the image
- WORKDIR – sets the working directory
- EXPOSE – documents the port used by the application
- CMD – specifies the default command

Result: A Dockerfile was created, a Docker image was successfully built, and the web application was run inside a container.

Practical 12: Demonstration and Execution of Basic Docker Commands

Aim: To demonstrate and execute commonly used Docker commands.

Step 1: Check Docker installation

```
docker --version
```

Step 2: Check Docker information

```
docker info
```

Shows information about Docker Engine.

Step 3: Download an image

```
docker pull hello-world
```

Step 4: View images

```
docker images
```

Step 5: Run a container

```
docker run hello-world
```

Step 6: View running containers

```
docker ps
```

Step 7: View all containers

```
docker ps -a
```

Step 8: Run an interactive Ubuntu container

```
docker run -it ubuntu bash
```

Inside the container, type exit to return to the host.

Step 9: Start a stopped container

```
docker start CONTAINER_ID
```

Replace CONTAINER_ID with the actual ID.

Step 10: Stop a container

```
docker stop CONTAINER_ID
```

Step 11: Remove a container

```
docker rm CONTAINER_ID
```

Step 12: Remove an image

```
docker rmi IMAGE_NAME
```

Only remove an image when it is no longer needed.

Step 13: View Docker networks

```
docker network ls
```

Step 14: View Docker volumes

```
docker volume ls
```

Important commands / notes

Most important commands:

```
docker pull IMAGE
```

```
docker images
```

```
docker run IMAGE
```

```
docker ps
```

```
docker ps -a
```

```
docker start CONTAINER_ID
```

```
docker stop CONTAINER_ID
```

```
docker rm CONTAINER_ID
```

```
docker rmi IMAGE
```

Result: Basic Docker commands were successfully demonstrated and executed.

Quick Revision – Most Important Commands

Git:

```
git init
git status
git add .
git commit -m "message"
git log --oneline
git branch
git switch branch
git merge branch
git clone URL
git remote -v
git push
git pull
```

GitHub/GitLab:

```
git remote add origin URL
git branch -M main
git push -u origin main
git clone URL
git switch -c feature
git push -u origin feature
```

Docker:

```
docker --version
docker pull IMAGE
docker images
docker build -t IMAGE .
docker run -d -p 8080:80 IMAGE
docker ps
docker ps -a
docker stop CONTAINER_ID
docker rm CONTAINER_ID
docker rmi IMAGE
```